

SMT流水线ECS接口调用说明书

文档概述

本文档基于SMT流水线系统的JSON文件和C#数据模型定义，详细说明了所有可用的API接口端点、请求参数、返回数据结构及JSON示例字段说明。开发者可根据本文档准确理解如何根据模型定义构造请求及处理响应数据。

目录

- 1. [接口概览](#)
- 2. [数据模型定义](#)
- 3. [命令接口](#)
- 4. [查询接口](#)
- 5. [回调接口](#)
- 6. [错误处理](#)
- 7. [完整示例代码](#)

1. 接口概览

1.1 系统概述

SMT流水线系统是一个自动化物料输送和扫码识别系统，主要用于SMT生产线上料箱的输送和扫码识别。系统包含多个扫码工位、入料工位和出料工位，支持与WES（仓储执行系统）的集成。

1.2 系统组成

- **扫码工位:** STATION_SCAN1~16（16个扫码位置）
- **入料工位:** STATION_INPUT1~4（4个入料位置）
- **出料工位:** STATION_OUTPUT1~4（4个出料位置）
- **流水线工位:** PIPELINE_INPUT2/4、PIPELINE_OUTPUT2/3/4

1.3 接口分类

接口类型	方向	说明
命令接口	WES → 设备	发送移动、取消等命令
查询接口	WES → 设备	查询设备状态
回调接口	设备 → WES	任务结果回传、事件推送

1.4 接口列表

序号	接口名称	HTTP方法	端点	说明
1	发送移动命令	POST	/api/v1/command	发送MOVE命令
2	取消命令	POST	/api/v1/command/cancel	取消正在执行的命令
3	查询设备状态	GET	/api/v1/status	查询所有设备状态
4	任务结果回传	POST	[WES回调地址]	设备向WES回传任务结果
5	事件推送	POST	[WES事件上报地址]	设备向WES推送事件

1.5 通信协议

- 协议: HTTP/1.1
- 数据格式: JSON
- 字符编码: UTF-8
- Content-Type: application/json

2. 数据模型定义

基于 Smt.PipeLine.Models.cs 文件，定义以下数据模型：

2.1 MoveCommand（移动命令）

完整模型定义：

```

public class MoveCommand
{
    [JsonPropertyName("device_id")]
    public string DeviceId { get; set; }           // 设备ID

    [JsonPropertyName("command_id")]
    public string CommandId { get; set; }         // 命令ID, 需全局唯一

    [JsonPropertyName("timestamp")]
    public string Timestamp { get; set; }         // 时间戳, 毫秒级

    [JsonPropertyName("task_type")]
    public string TaskType { get; set; }          // 任务类型: MOVE_FORWARD/MOVE_BACKWARD/MOVE_L

    [JsonPropertyName("priority")]
    public int Priority { get; set; }              // 优先级 (1-10, 10最高)

    [JsonPropertyName("timeout")]
    public int Timeout { get; set; }              // 期望完成时间(ms)

    [JsonPropertyName("source")]
    public MoveLocationInfo Source { get; set; }   // 源位置信息
}

```

2.2 MoveLocationInfo (移动位置信息)

```

public class MoveLocationInfo
{
    [JsonPropertyName("location_id")]
    public string LocationId { get; set; }         // 位置ID

    [JsonPropertyName("location_type")]
    public string LocationType { get; set; }       // 工位类型: SCAN_PLATFORM/PICK_PLATFORM/PUT_P

    [JsonPropertyName("bin_id")]
    public string BinId { get; set; }              // 箱子ID, ECS追溯用, 可选

    [JsonPropertyName("bin_type")]
    public string BinType { get; set; }            // 箱子类型: 三格箱/六格箱
}

```

2.3 DeviceStatusItem (设备状态项)

```
public class DeviceStatusItem
{
    [JsonPropertyName("device_id")]
    public string DeviceId { get; set; }           // 设备ID

    [JsonPropertyName("timestamp")]
    public long Timestamp { get; set; }           // 时间戳(毫秒级)

    [JsonPropertyName("current_cmd_id")]
    public string CurrentCmdId { get; set; }       // 当前正在执行的命令ID，空闲时为空

    [JsonPropertyName("error_code")]
    public string ErrorCode { get; set; }          // 错误代码，无错误时为"NONE"

    [JsonPropertyName("status")]
    public string Status { get; set; }             // 设备状态
}
```

2.4 EventPush (事件推送)

```
public class EventPush
{
    [JsonPropertyName("device_id")]
    public string DeviceId { get; set; }           // 设备ID

    [JsonPropertyName("timestamp")]
    public long Timestamp { get; set; }           // 事件时间戳(毫秒级)

    [JsonPropertyName("event_type")]
    public string EventType { get; set; }          // 事件类型：BIN_ARRIVED/BIN_DEPARTED/SCAN_COM

    [JsonPropertyName("data")]
    public EventData Data { get; set; }            // 业务负载数据(可选)
}
```

2.5 EventData (事件数据)

```
public class EventData
{
    [JsonPropertyName("location")]
    public string Location { get; set; }           // 位置

    [JsonPropertyName("barcode")]
    public string Barcode { get; set; }           // 条码信息
}
```

2.6 TaskResultReport (任务结果报告)

```
public class TaskResultReport
{
    [JsonPropertyName("command_id")]
    public string CommandId { get; set; }         // 必须回传原指令ID

    [JsonPropertyName("device_id")]
    public string DeviceId { get; set; }         // 设备ID

    [JsonPropertyName("result")]
    public string Result { get; set; }           // 执行结果: SUCCESS 或 FAILED

    [JsonPropertyName("finish_time")]
    public long FinishTime { get; set; }         // 完成时间戳(毫秒级)

    [JsonPropertyName("data")]
    public TaskResultData Data { get; set; }     // 业务回传数据(可选)

    [JsonPropertyName("error_detail")]
    public ErrorDetail ErrorDetail { get; set; } // 错误详情(result=FAILED时必填)
}
```

2.7 ApiResult (API响应)

```
public class ApiResult
{
    [JsonPropertyName("status")]
    public string Status { get; set; }           // 状态: SUCCESS 或 FAILED

    [JsonPropertyName("message")]
    public string Message { get; set; }         // 消息: 成功或错误描述

    [JsonPropertyName("code")]
    public string Code { get; set; }            // 状态码: 200成功, 400参数错误, 503设备忙/故障

    [JsonPropertyName("trace_id")]
    public string TraceId { get; set; }         // 可选, 供应商内部日志ID-附带时间戳或command_id
}
```

3. 命令接口

3.1 发送移动命令

接口描述: 向指定的流水线工位发送移动命令, 控制料箱移动。

3.1.1 请求说明

HTTP方法: POST

请求端点: /api/v1/command

请求头:

Content-Type: application/json

3.1.2 请求参数

参数名	类型	必填	说明	示例值
device_id	string	是	设备ID, 指定执行命令的工位	"STATION_SCAN1"

参数名	类型	必填	说明	示例值
command_id	string	是	命令ID，需全局唯一	"CMD-20251215-1001"
timestamp	string	是	时间戳，毫秒级	"1702627300000"
task_type	string	是	任务类型	"MOVE_FORWARD"
priority	number	是	优先级(1-10)，10最高	1
timeout	number	是	期望完成时间(ms)	30000
source	object	是	源位置信息，见 MoveLocationInfo	-

3.1.3 task_type 枚举值

值	说明
MOVE_FORWARD	前进
MOVE_BACKWARD	后退
MOVE_LEFT	左移
MOVE_RIGHT	右移

3.1.4 location_type 枚举值

值	说明
SCAN_PLATFORM	扫码平台
PICK_PLATFORM	取料平台
PUT_PLATFORM	放料平台

3.1.5 请求示例

场景: 从扫码平台1前进移动

```
{
  "device_id": "STATION_SCAN1",
  "command_id": "CMD-20251215-1001",
  "timestamp": "1702627300000",
  "task_type": "MOVE_FORWARD",
  "priority": 1,
  "timeout": 30000,
  "source": {
    "location_id": "STATION_SCAN1",
    "location_type": "SCAN_PLATFORM",
    "bin_id": "BIN_104",
    "bin_type": "三格箱"
  }
}
```

字段说明:

- device_id : "STATION_SCAN1" - 指定扫码工位1
- task_type : "MOVE_FORWARD" - 前进移动
- priority : 1 - 优先级1
- timeout : 30000 - 超时时间30秒
- source : 源位置信息
 - location_id : "STATION_SCAN1" - 当前工位ID
 - location_type : "SCAN_PLATFORM" - 工位类型为扫码平台
 - bin_id : "BIN_104" - 料箱ID (ECS追溯用)
 - bin_type : "三格箱" - 料箱类型

3.1.6 响应说明

响应结构: ApiResult

字段名	类型	说明
status	string	状态: "SUCCESS" 或 "FAILED"
message	string	消息: "Accepted" 或错误描述
code	string	状态码: 200成功接收, 400参数错误, 503设备忙/故障
trace_id	string	可选, 供应商内部日志ID-附带时间戳或command_id

message枚举值:

- Accepted：命令已被接收
- Rejected：命令被拒绝
- Timeout：超时
- DeviceBusy：设备忙碌
- DeviceError：设备错误

成功响应示例:

```
{
  "status": "SUCCESS",
  "message": "Accepted",
  "code": "200",
  "trace_id": "DEV-LOG-CMD-20251215-1001"
}
```

失败响应示例:

```
{
  "status": "FAILED",
  "message": "DeviceBusy",
  "code": "503",
  "trace_id": "DEV-LOG-CMD-20251215-1001"
}
```

3.2 取消命令

接口描述: 取消正在执行的命令。

3.2.1 请求说明

HTTP方法: POST

请求端点: /api/v1/command/cancel

3.2.2 请求参数

参数名	类型	必填	说明	示例值
command_id	string	是	要取消的命令ID	"CMD-20251215-1001"

3.2.3 请求示例

```
{  
  "command_id": "CMD-20251215-1001"  
}
```

3.2.4 响应说明

成功响应示例:

```
{  
  "status": "SUCCESS",  
  "message": "Cancelled",  
  "code": "200",  
  "trace_id": "DEV-LOG-CMD-20251215-1001"  
}
```

失败响应示例:

```
{  
  "status": "FAILED",  
  "message": "Command not found or already completed",  
  "code": "400",  
  "trace_id": "DEV-LOG-CMD-20251215-1001"  
}
```

4. 查询接口

4.1 查询设备状态

接口描述: 查询所有流水线工位的当前状态。

4.1.1 请求说明

HTTP方法: GET

请求端点: /api/v1/status

4.1.2 请求参数

无请求参数。

4.1.3 响应说明

响应结构: PipelineDeviceStatusResponse

字段名	类型	说明
status	array	设备状态数组，每个元素包含一个设备的状态信息

DeviceStatus对象:

字段名	类型	说明	示例值
device_id	string	设备ID	"STATION_SCAN1"
timestamp	number	时间戳(毫秒级)	1702627300000
current_cmd_id	string	当前正在执行的命令ID，空闲时为空	"CMD-20251215-1001"
error_code	string	错误代码，无错误时为"NONE"	"NONE"
status	string	设备状态	"RUNNING"

设备状态枚举值:

值	说明
IDLE	空闲
RUNNING	运行中
ERROR	错误
PAUSED	暂停
STOPPED	已停止
OFFLINE	离线

4.1.4 响应示例

```
{
  "status": [
    {
      "device_id": "STATION_SCAN1",
      "timestamp": 1702627300000,
      "current_cmd_id": "CMD-20251215-1001",
      "error_code": "NONE",
      "status": "RUNNING"
    },
    {
      "device_id": "STATION_SCAN2",
      "timestamp": 1702627300000,
      "current_cmd_id": "CMD-20251215-1002",
      "error_code": "NONE",
      "status": "RUNNING"
    },
    {
      "device_id": "STATION_INPUT1",
      "timestamp": 1702627300000,
      "current_cmd_id": "CMD-20251215-1017",
      "error_code": "NONE",
      "status": "RUNNING"
    },
    {
      "device_id": "STATION_OUTPUT1",
      "timestamp": 1702627300000,
      "current_cmd_id": "CMD-20251215-1021",
      "error_code": "NONE",
      "status": "RUNNING"
    }
  ]
}
```

字段说明:

- 返回多个设备的状态数组（包括扫码工位、入料工位、出料工位）
- device_id : 设备唯一标识符
 - 扫码工位: STATION_SCAN1~16
 - 入料工位: STATION_INPUT1~4
 - 出料工位: STATION_OUTPUT1~4

- current_cmd_id：如果设备正在执行命令，返回command_id；如果空闲，该字段为null或空字符串
- error_code："NONE"表示无错误，其他值表示具体错误类型
- status："RUNNING"表示设备正在运行命令

5. 回调接口

回调接口由设备主动向WES（仓储执行系统）发起，用于回传任务执行结果或推送事件。

5.1 任务结果回传（Report Result）

接口描述：设备完成移动命令后，向WES回传任务执行结果。

5.1.1 请求说明

HTTP方法：POST

请求端点：[WES提供的回调地址]

5.1.2 请求参数

参数名	类型	必填	说明	示例值
command_id	string	是	必须回传原指令ID	"CMD-20251215-1001"
device_id	string	是	设备ID	"STATION_SCAN1"
result	string	是	执行结果: SUCCESS 或 FAILED	"SUCCESS"
finish_time	number	是	完成时间戳(毫秒级)	1702627250000
data	object	否	业务回传数据	-
error_detail	object	否	错误详情(result=FAILED时必填)	-

data对象字段：

字段名	类型	说明	示例值
actual_qty	number	实际操作数量	1
result	string	移动指令具体结果	"MOVE_FINISHED"

result枚举值:

值	说明
MOVE_FINISHED	移动完成
MOVE_FAILED	移动失败

error_detail对象字段:

字段名	类型	说明	示例值
error_code	string	错误码，0为无错误	"0"
error_message	string	具体错误信息	""

5.1.3 请求示例

成功回传示例:

```
{
  "command_id": "CMD-20251215-1001",
  "device_id": "STATION_SCAN1",
  "result": "SUCCESS",
  "finish_time": 1702627250000,
  "data": {
    "actual_qty": 1,
    "result": "MOVE_FINISHED"
  }
}
```

失败回传示例:

```
{
  "command_id": "CMD-20251215-1001",
  "device_id": "STATION_SCAN1",
  "result": "FAILED",
  "finish_time": 1702627250000,
  "data": {
    "actual_qty": 0,
    "result": "MOVE_FAILED"
  },
  "error_detail": {
    "error_code": "1001",
    "error_message": "Path blocked"
  }
}
```

字段说明:

- `command_id`: 必须与原发送的命令ID一致, 用于任务追踪
- `result`: "SUCCESS"表示命令执行成功, "FAILED"表示执行失败
- `finish_time`: 任务完成的精确时间戳
- `data.actual_qty`: 实际完成的移动数量
- `data.result`: 具体执行结果, 成功时为"MOVE_FINISHED"
- `error_detail`: 仅在`result="FAILED"`时需要提供, 说明失败原因

5.1.4 响应说明

WES接收到回调后, 需要返回确认响应。

成功响应示例:

```
{
  "status": "SUCCESS",
  "message": "ACK",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

失败响应示例:

```
{
  "status": "FAILED",
  "message": "Invalid parameters",
  "code": "400",
  "trace_id": "DEV-LOG-1702627300000"
}
```

5.2 事件推送（Event Push）

接口描述: 料箱到达、离开或扫码设备完成扫码后，向WES推送事件。

5.2.1 请求说明

HTTP方法: POST

请求端点: [WES提供的回调地址]

5.2.2 请求参数

参数名	类型	必填	说明	示例值
device_id	string	是	设备ID	"STATION_SCAN1"
event_type	string	是	事件类型	"SCAN_COMPLETED"
timestamp	number	是	事件时间戳(毫秒级)	1702627300000
data	object	否	业务负载数据	-

event_type枚举值:

值	说明
BIN_ARRIVED	料箱到达
BIN_DEPARTED	料箱离开
SCAN_COMPLETED	扫码完成

data对象字段:

字段名	类型	说明	示例值
location	string	位置	"STATION_SCAN1"
barcode	string	条码信息	"A" 或 "ERROR"

5.2.3 请求示例

扫码完成事件示例（有扫码设备）：

```
{
  "device_id": "STATION_SCAN1",
  "event_type": "SCAN_COMPLETED",
  "timestamp": 1702627300000,
  "data": {
    "location": "STATION_SCAN1",
    "barcode": "A"
  }
}
```

扫码完成事件示例（扫码失败）：

```
{
  "device_id": "STATION_SCAN1",
  "event_type": "SCAN_COMPLETED",
  "timestamp": 1702627300000,
  "data": {
    "location": "STATION_SCAN1",
    "barcode": "ERROR"
  }
}
```

扫码完成事件示例（有完整条码）：

```
{
  "device_id": "STATION_SCAN5",
  "event_type": "SCAN_COMPLETED",
  "timestamp": 1702627300000,
  "data": {
    "location": "STATION_SCAN5",
    "barcode": "Barcode1234567890-A"
  }
}
```

料箱到达事件示例（无扫码设备）：

```
{
  "device_id": "STATION_INPUT1",
  "event_type": "BIN_ARRIVED",
  "timestamp": 1702627300000,
  "data": {
    "location": "STATION_INPUT1",
    "barcode": ""
  }
}
```

料箱离开事件示例（无扫码设备）：

```
{
  "device_id": "STATION_OUTPUT1",
  "event_type": "BIN_DEPARTED",
  "timestamp": 1702627300000,
  "data": {
    "location": "STATION_OUTPUT1",
    "barcode": ""
  }
}
```

字段说明：

- device_id：设备ID
 - 扫码工位: STATION_SCAN1~16
 - 入料工位: STATION_INPUT1~4、PIPELINE_INPUT2/4
 - 出料工位: STATION_OUTPUT1~4、PIPELINE_OUTPUT2/3/4

- event_type：事件类型
 - "SCAN_COMPLETED": 扫码完成（扫码工位）
 - "BIN_ARRIVED": 料箱到达（入料工位）
 - "BIN_DEPARTED": 料箱离开（出料工位）
- timestamp：事件发生的时间戳
- data.location：事件发生的具体位置
- data.barcode：扫码数据
 - 有扫码设备时：扫描到的条码数据或"ERROR"（扫码失败）
 - 无扫码设备时：空字符串

5.2.4 响应说明

WES接收到事件推送后，需要返回确认响应。

成功响应示例：

```
{
  "status": "SUCCESS",
  "message": "ACK",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

6. 错误处理

6.1 HTTP状态码

状态码	说明	处理建议
200	成功	正常处理响应数据
400	请求参数错误	检查请求参数格式和必填字段
503	设备忙/故障	等待设备空闲后重试或联系技术支持

6.2 业务错误码

code	说明	处理建议
200	成功	-
400	参数错误	检查参数格式
503	设备忙	等待后重试

6.3 设备错误码 (error_code)

error_code	说明	处理建议
NONE	无错误	-
1001	路径被阻挡	清理障碍物后重试
1002	移动失败	检查位置是否可访问
1003	设备故障	联系技术支持

6.4 错误处理示例

```
public async Task<bool> SendCommandWithErrorHandling(MoveCommand command, int maxRetries = 3)
{
    int retryCount = 0;

    while (retryCount < maxRetries)
    {
        try
        {
            var result = await SendCommandAsync(command);

            if (result.Status == "SUCCESS" && result.Code == "200")
            {
                return true;
            }

            // 设备忙碌，等待后重试
            if (result.Code == "503")
            {
                Console.WriteLine($"设备忙碌，等待后重试... (第{retryCount + 1}次)");
                await Task.Delay(5000);
                retryCount++;
                continue;
            }

            // 参数错误，不重试
            if (result.Code == "400")
            {
                Console.WriteLine($"参数错误: {result.Message}");
                return false;
            }

            // 其他错误，不重试
            Console.WriteLine($"命令发送失败: {result.Message} (错误码: {result.Code})");
            return false;
        }
        catch (Exception ex)
        {
            retryCount++;
            Console.WriteLine($"发送命令异常: {ex.Message} (第{retryCount}次)");

            if (retryCount >= maxRetries)
```

```
{  
    Console.WriteLine("已达到最大重试次数，放弃发送");  
    return false;  
}  
  
    await Task.Delay(5000);  
}  
}  
  
return false;  
}
```

7. 完整示例代码

7.1 API客户端实现

```
using System;
using System.Net.Http;
using System.Text;
using System.Text.Json;
using System.Threading.Tasks;
using Smt.PipeLine.Models;

namespace Smt.PipeLine.Client
{
    public class SmtPipelineApiClient
    {
        private readonly HttpClient _httpClient;
        private readonly string _baseUrl;

        public SmtPipelineApiClient(string baseUrl)
        {
            _baseUrl = baseUrl;
            _httpClient = new HttpClient
            {
                BaseAddress = new Uri(baseUrl)
            };
            _httpClient.DefaultRequestHeaders.Accept.Add(
                new System.Net.Http.Headers.MediaTypeWithQualityHeaderValue("application/json"));
        }

        /// <summary>
        /// 发送移动命令
        /// </summary>
        public async Task<ApiResult> SendMoveCommandAsync(MoveCommand command)
        {
            var json = JsonSerializer.Serialize(command);
            var content = new StringContent(json, Encoding.UTF8, "application/json");

            var response = await _httpClient.PostAsync("/api/v1/command", content);
            var responseContent = await response.Content.ReadAsStringAsync();

            return JsonSerializer.Deserialize<ApiResult>(responseContent);
        }
    }
}
```

```

/// <summary>
/// 取消命令
/// </summary>
public async Task<ApiResult> CancelCommandAsync(string commandId)
{
    var request = new CancelCommandRequest { CommandId = commandId };
    var json = JsonSerializer.Serialize(request);
    var content = new StringContent(json, Encoding.UTF8, "application/json");

    var response = await _httpClient.PostAsync("/api/v1/command/cancel", content);
    var responseContent = await response.Content.ReadAsStringAsync();

    return JsonSerializer.Deserialize<ApiResult>(responseContent);
}

/// <summary>
/// 查询设备状态
/// </summary>
public async Task<PipelineDeviceStatusResponse> GetDeviceStatusAsync()
{
    var response = await _httpClient.GetAsync("/api/v1/status");
    var content = await response.Content.ReadAsStringAsync();

    return JsonSerializer.Deserialize<PipelineDeviceStatusResponse>(content);
}

/// <summary>
/// 创建前进移动命令
/// </summary>
public MoveCommand CreateMoveForwardCommand(
    string deviceId, string locationId, string binId, string binType)
{
    return new MoveCommand
    {
        DeviceId = deviceId,
        CommandId = $"CMD-{DateTime.Now:yyyyMMdd-HH:mm:ss}",
        Timestamp = DateTimeOffset.Now.ToUnixTimeMilliseconds().ToString(),
        TaskType = TaskTypeEnum.MOVE_FORWARD,
        Priority = 1,
        Timeout = 30000,
        Source = new MoveLocationInfo
        {

```

```

        LocationId = locationId,
        LocationType = LocationTypeEnum.SCAN_PLATFORM,
        BinId = binId,
        BinType = binType
    }
};
}

/// <summary>
/// 创建后退移动命令
/// </summary>
public MoveCommand CreateMoveBackwardCommand(
    string deviceId, string locationId, string binId, string binType)
{
    return new MoveCommand
    {
        DeviceId = deviceId,
        CommandId = $"CMD-{DateTime.Now:yyyyMMdd-HH:mm:ss}",
        Timestamp = DateTimeOffset.Now.ToUnixTimeMilliseconds().ToString(),
        TaskType = TaskTypeEnum.MOVE_BACKWARD,
        Priority = 1,
        Timeout = 30000,
        Source = new MoveLocationInfo
        {
            LocationId = locationId,
            LocationType = LocationTypeEnum.SCAN_PLATFORM,
            BinId = binId,
            BinType = binType
        }
    };
}

/// <summary>
/// 创建左移命令
/// </summary>
public MoveCommand CreateMoveLeftCommand(
    string deviceId, string locationId, string binId, string binType)
{
    return new MoveCommand
    {
        DeviceId = deviceId,
        CommandId = $"CMD-{DateTime.Now:yyyyMMdd-HH:mm:ss}",
        Timestamp = DateTimeOffset.Now.ToUnixTimeMilliseconds().ToString(),

```

```

        TaskType = TaskTypeEnum.MOVE_LEFT,
        Priority = 1,
        Timeout = 30000,
        Source = new MoveLocationInfo
        {
            LocationId = locationId,
            LocationType = LocationTypeEnum.SCAN_PLATFORM,
            BinId = binId,
            BinType = binType
        }
    };
}

/// <summary>
/// 创建右移命令
/// </summary>
public MoveCommand CreateMoveRightCommand(
    string deviceId, string locationId, string binId, string binType)
{
    return new MoveCommand
    {
        DeviceId = deviceId,
        CommandId = $"CMD-{DateTime.Now:yyyyMMdd-HH:mm:ss}",
        Timestamp = DateTimeOffset.Now.ToUnixTimeMilliseconds().ToString(),
        TaskType = TaskTypeEnum.MOVE_RIGHT,
        Priority = 1,
        Timeout = 30000,
        Source = new MoveLocationInfo
        {
            LocationId = locationId,
            LocationType = LocationTypeEnum.SCAN_PLATFORM,
            BinId = binId,
            BinType = binType
        }
    };
}
}
}
}
}

```

7.2 使用示例

```
using System;
using System.Threading.Tasks;
using Smt.PipeLine.Client;
using Smt.PipeLine.Models;

class Program
{
    static async Task Main(string[] args)
    {
        // 初始化API客户端
        var client = new SmtPipelineApiClient("http://192.168.1.100:8080");

        // ===== 示例1: 查询设备状态 =====
        Console.WriteLine("查询设备状态...");
        var statusArray = await client.GetDeviceStatusAsync();
        foreach (var status in statusArray.Status)
        {
            Console.WriteLine($"设备 {status.DeviceId}: 状态={status.Status}, 当前命令={status.Ci
        }

        // ===== 示例2: 发送前进移动命令 =====
        Console.WriteLine("\n发送前进移动命令...");

        var moveForward = client.CreateMoveForwardCommand(
            deviceId: DeviceIdConstants.STATION_SCAN1,
            locationId: "STATION_SCAN1",
            binId: "BIN_104",
            binType: BinTypeEnum.THREE_CELL
        );

        var result1 = await client.SendMoveCommandAsync(moveForward);
        Console.WriteLine($"前进移动命令结果: {result1.Status} - {result1.Message} (Code: {result1

        // ===== 示例3: 发送左移命令 =====
        Console.WriteLine("\n发送左移命令...");

        var moveLeft = client.CreateMoveLeftCommand(
            deviceId: DeviceIdConstants.STATION_SCAN2,
            locationId: "STATION_SCAN2",
            binId: "BIN_105",
            binType: BinTypeEnum.SIX_CELL
```

```
);

var result2 = await client.SendMoveCommandAsync(moveLeft);
Console.WriteLine($"左移命令结果: {result2.Status} - {result2.Message} (Code: {result2.C

// ===== 示例4: 取消命令 =====
Console.WriteLine("\n取消命令示例...");
var cancelResult = await client.CancelCommandAsync(moveForward.CommandId);
Console.WriteLine($"取消结果: {cancelResult.Status} - {cancelResult.Message}");
}
}
```

8. 附录

8.1 设备ID清单

8.1.1 扫码工位（16个）

设备ID	说明
STATION_SCAN1	扫码工位1
STATION_SCAN2	扫码工位2
STATION_SCAN3	扫码工位3
STATION_SCAN4	扫码工位4
STATION_SCAN5	扫码工位5
STATION_SCAN6	扫码工位6
STATION_SCAN7	扫码工位7
STATION_SCAN8	扫码工位8
STATION_SCAN9	扫码工位9
STATION_SCAN10	扫码工位10
STATION_SCAN11	扫码工位11

设备ID	说明
STATION_SCAN12	扫码工位12
STATION_SCAN13	扫码工位13
STATION_SCAN14	扫码工位14
STATION_SCAN15	扫码工位15
STATION_SCAN16	扫码工位16

8.1.2 入料工位（6个）

设备ID	说明
STATION_INPUT1	入料工位1
STATION_INPUT2	入料工位2
STATION_INPUT3	入料工位3
STATION_INPUT4	入料工位4
PIPELINE_INPUT2	流水线入料工位2
PIPELINE_INPUT4	流水线入料工位4

8.1.3 出料工位（7个）

设备ID	说明
STATION_OUTPUT1	出料工位1
STATION_OUTPUT2	出料工位2
STATION_OUTPUT3	出料工位3
STATION_OUTPUT4	出料工位4
PIPELINE_OUTPUT2	流水线出料工位2
PIPELINE_OUTPUT3	流水线出料工位3
PIPELINE_OUTPUT4	流水线出料工位4

8.2 事件类型对照表

event_type	适用设备	触发时机	barcode字段
SCAN_COMPLETED	扫码工位 (STATION_SCAN1~16)	扫码完成	扫码结果或"ERROR"
BIN_ARRIVED	入料工位 (STATION_INPUT1~4, PIPELINE_INPUT2/4)	料箱到达	空字符串
BIN_DEPARTED	出料工位 (STATION_OUTPUT1~4, PIPELINE_OUTPUT2/3/4)	料箱离开	空字符串

8.3 术语表

术语	英文	说明
WES	Warehouse Execution System	仓储执行系统
ECS	Enterprise Control System	企业控制系统
MOVE_FORWARD	Move Forward	前进移动
MOVE_BACKWARD	Move Backward	后退移动
MOVE_LEFT	Move Left	左移
MOVE_RIGHT	Move Right	右移
BIN_ARRIVED	Bin Arrived	料箱到达事件
BIN_DEPARTED	Bin Departed	料箱离开事件
SCAN_COMPLETED	Scan Completed	扫码完成事件
TRACE_ID	Trace ID	用于日志追踪的唯一标识

8.4 料箱类型

类型	说明
三格箱	三格料箱

类型	说明
六格箱	六格料箱

9. 版本历史

版本	日期	说明
1.0.0	2025-01-15	初始版本

文档结束